A.

This report will help a business with a problem of maximizing or minimizing their inventory and revenue of rented films.

The business report will show the most rented movies, least rented movies, and the total rental days. It will show each rental ID and count it as a rental. It will also show movies that are not rented or rented the least.

This will show popularity trends which will allow management to order more copies of movies that are in demand. It will also show movies underperforming and that may need to be removed from stock. Tracking the number of days rented will show customer usage patterns. This can show movies or categories that are consistently kept longer. The rental business can adjust pricing to maximize revenue based on popularity of movies. The business report will show no rentals and help the business decide where to discontinue them or keep them. It could free up space for newer or more popular movies. This will help the business identify where they can adjust prices but also maximize inventory.

1.

The summary table will include the following fields:

- film_id (INTEGER) PRIMARY KEY
- total_rentals (INTEGER)
- actual_rental_days (INTEGER)
- title (VARCHAR(255))

The detailed table will consist of:

- film_id (INTEGER)
- title (VARCHAR(255))
- rental_id (INTEGER)
- rental_date (TIMESTAMP)
- return_date (TIMESTAMP)
- inventory_id (INTEGER)
- actual_rental_days (INTEGER)

2. The data fields used for the report will VARCHAR, INTEGER, TIMESTAMP.

3. The tables used from the given dataset that will provide the data are the film table, rental table, and inventory table.

4. One field in my detailed table section that will require transformation with a user-defined function will the rental_date and return_date to acutal_rental_days. The function will take the difference between the return_date and rental_date in this function. This report needs to have the actual dates the film was rented for the summary table to have the total days rented.

5. The detail table will give raw data for each film. This will show information for each film and help dispute any issues with renting the movie by having every single transaction but also help with any auditing.

The summary table will give aggregated data showing popularity of a movie and help to increase or decrease inventory of the movies.

6. The report should be refreshed monthly. This will allow the store to discard movies that are not rented or not as popular. This will allow the business to make room for new movies.

B.

```
CREATE OR REPLACE FUNCTION rental_days(start_date TIMESTAMP, end_date TIMESTAMP)
RETURNS INTEGER AS $$
BEGIN
IF start_date IS NULL OR end_date IS NULL THEN
   RETURN 0;
   END IF;
   RETURN EXTRACT(day FROM (end_date - start_date));
END;
$$ LANGUAGE plpgsql;
```

C.

```
CREATE TABLE detail_table (
        film_id INTEGER,
        title VARCHAR(255),
        rental_id INTEGER,
```

```
        rental_date TIMESTAMP,
        return_date TIMESTAMP,
        inventory_id INTEGER,
        actual_rental_days INTEGER
);

CREATE TABLE summary_table (
    film_id INTEGER PRIMARY KEY,
    title VARCHAR(255),
    total_rentals INTEGER,
    total_rental_days INTEGER
);
```

D.

```
INSERT INTO detail_table(film_id, title, rental_id, rental_date, return_date, inventory_id, actual_rental_days)
SELECT
f.film_id,
f.title,
r.rental_id,
r.rental_date,
r.return_date,
d.inventory_id,
rental_days (r.rental_date, r.return_date) AS actual_rental_days
```

```
FROM film f
LEFT JOIN inventory d on f.film_id = d.film_id
LEFT JOIN rental r on d.inventory_id = r.inventory_id;


INSERT INTO summary_table(film_id, title, total_rentals, total_rental_days)
SELECT
   f.film_id,
   f.title,
   COUNT(d.rental_id) AS total_rentals,
   COALESCE(SUM(d.actual_rental_days), 0) AS total_rental_days
FROM film f
LEFT JOIN detail_table d ON f.film_id = d.film_id
GROUP BY f.film_id, f.title
ORDER BY total_rentals DESC, total_rental_days DESC;
```

E.

```
CREATE OR REPLACE FUNCTION update_summary_on_detail_insert()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
  INSERT INTO summary_table (film_id, title, total_rentals, total_rental_days)
  SELECT
     f.film_id,
     f.title,
     COUNT(d.rental_id) AS total_rentals,
```

```
    SUM(COALESCE(d.actual_rental_days,0)) AS total_rental_days
  FROM film f
  LEFT JOIN detail_table d
        ON f.film_id = d.film_id
  WHERE f.film_id = NEW.film_id
  GROUP BY f.film_id, f.title
  ON CONFLICT (film_id) DO UPDATE
  SET total_rentals = EXCLUDED.total_rentals,
    total_rental_days = EXCLUDED.total_rental_days;
  RETURN NEW;
END;
$$;

CREATE TRIGGER detail_insert
AFTER INSERT ON detail_table
FOR EACH ROW
EXECUTE FUNCTION update_summary_on_detail_insert();
```

F. This can be run with a job scheduler like pgAgent to run the code and automate it to run the refresh_data procedure at least monthly.

```
CREATE OR REPLACE PROCEDURE refresh_data()
LANGUAGE plpgsql
AS $$
BEGIN
TRUNCATE detail_table CASCADE;
TRUNCATE summary_table CASCADE;
```

```
ALTER TABLE detail_table DISABLE TRIGGER detail_insert;

INSERT INTO detail_table(film_id, title, rental_id, rental_date, return_date, inventory_id,
actual_rental_days)

SELECT

f.film_id,

f.title,

r.rental_id,

r.rental_date,

r.return_date,

d.inventory_id,

rental_days (r.rental_date, r.return_date) AS actual_rental_days


FROM film f

LEFT JOIN inventory d on f.film_id = d.film_id

LEFT JOIN rental r on d.inventory_id = r.inventory_id;


ALTER TABLE detail_table ENABLE TRIGGER detail_insert;

INSERT INTO summary_table(film_id, title, total_rentals, total_rental_days)

SELECT

   f.film_id,

   f.title,

   COUNT(d.rental_id) AS total_rentals,

   COALESCE(SUM(d.actual_rental_days), 0) AS total_rental_days

FROM film f

LEFT JOIN detail_table d ON f.film_id = d.film_id

GROUP BY f.film_id, f.title
```

```
ORDER BY total_rentals DESC, total_rental_days DESC;


END;
$$;


CALL refresh_data();
```

No other sources were used to make this code other than what was provided in the database for the class.

Panopto video link:

https://wgu.hosted.panopto.com/Panopto/Pages/Viewer.aspx?id=1984529a-42be-4876-bf7a-b3520013db8e